FreeBSD Jails Orchestration with Ansible

Tutorial with examples

Vladimir Botka

Contents

FreeBSD Jails Orchestration with Ansible

Presenter Notes.

These notes can be downloaded from https://github.com/vbotka/ansible-collection-freebsd/blob/devel/docs/presentation/notes.md

We start with a brief overview of Ansible and FreeBSD. Then, we will review the proposed FreeBSD collection with the focus on the iocage plugins.

The main part of this tutorial are examples. There are approx. 30 of them in 4 categories:

- installation and configuration of an iocage host
- usage of the iocage plugins
- creation of FreeBSD jails as Ansible remote hosts
- orchestration of the FreeBSD jails by Ansible.

In the current scope of this tutorial, the orchestration means automated management of jails across a cluster of machines. We want to conclude this tutorial with a project described in the dictionary project

The main goal is to present the idea that FreeBSD Ansible collection is needed and propose how to proceed to create such a collection.

This presentation does not cover the use case of FreeBSD as an Ansible

Table of Contents

1. Introduction
2. FreeBSD collection
3. Examples
4. Install configure and activate iocage
5. Plugins iocage
6. Ansible client
7. Infrastructure
8. Appendix

Section 1

Introduction

I'm not sure how many time we will have for the questions at the end of the presentation. Therefore, do not hesitate and ask immediately.

(5) Why Ansible?

- There are many system management tools. We decided to pick Ansible because it is the leading tool.
- AI generated code. Can be useful, but you have to know what you want. I other words, you have to ask good questions. This presentation is to help ask good questions.
  AI is not a subject of this presentation. But, still, what is the latest Ansible AI status?
  At the moment it can be described as `IT automation with agentic AI`.
  1) Introducing the `Model Context Protocol` MCP server for `Red Hat Ansible Automation Platform` (technology preview)
     MCP for Ansible Deploying MCP AAP MCP Service
  2) The integration of Ansible, `Model Context Protocol` (MCP), and `ARA` Records enables `AI-powered, conversational monitoring and troubleshooting of Ansible playbooks`. See: ARA "ARA Records Ansible and makes it easier to understand and troubleshoot."
  3) Ansible Development Tools MCP Server

Note: MCP (Model Context Protocol) is an open standard that enables AI models to use external AI tools and services via a unified interface. Using the Ansible MCP server, you can connect your Ansible Automation Platform with your preferred external AI tool (such as Claude, Cursor, or Chat-GPT).

- To be informed about latest development join Ansible Forum. This is the

(6) Does Ansible work with FreeBSD?

When you decide to use Ansible to manage FreeBSD, important question is: Do you need your Ansible code also cover other systems in parallel to FreeBSD? This means, do you have to write your playbooks, roles, and plugins to manage also other systems in parallel to FreeBSD? This question is crucial. The code is much simpler if you cover FreeBSD only. In this presentation, we focus on the FreeBSD management by Ansible.
You can write your own FreeBSD playbooks, roles, and plugins.
(TBD. Simple module example)
Goals of this presentation:

- try to analyze FreeBSD specifics
- try to explain Ansible options
- explain the current status of the proposed FreeBSD collection.

There are problems with some Ansible modules running on FreeBSD. The modules `service` and `sysctl` are typical examples.
ansible.builtin.service
FreeBSD service arguments are passed in the wrong order #85156 service: Fix order of arguments in FreeBSD #85190 (open since May, 2025)
Quoting last comment on Oct 2, 2025 conversation

> *A different solution I would personally accept is a new, FreeBSD-specific module and a note on the generic service module that it does not accept*

(7) Ansible collections

- An important aspect in the complexity: To make qualified decision in Ansible, multiple areas of expertise are needed:
  - Multiple OS. Most modules are created for Linux.
  - OS administration experience.
  - Multiple programming languages. Logical consequences are collections focused on particular technologies.

There are more than 100 collections included in the Ansible distribution, sometimes referenced as 'battery included'. See Collection Index

Ansible introduced the collections in the version 2.8, released in the year 2019. This feature provided a new distribution format for Ansible content, including playbooks, roles, and plugins.

It's necessary to mention that Ansible is highly configurable. Everything is a plugin. See Working with plugins.

Later we will describe our iocage plugins:

- iocage inventory plugin,
- iocage module, and
- iocage filter

The introduction of collections allowed for several key changes:

- Independent release cycles: Content maintainers could develop and release their plugins outside of the main Ansible core release cycle, allowing for quicker bug fixes and feature additions.

(8) Ansible collections ansible.*

Take a look at who the authors of the collections are. This can tell you what the collection support level might be.
Take a look at what FreeBSD version(s) are being tested.
Quoting ansible-core 2.20 ... Release Notes

> *ansible-test - Replace FreeBSD 14.2 with 14.3. ansible-test - Use OS packages to satisfy controller requirements on FreeBSD 13.5 ...*

```
shell> git remote -v
origin  https://github.com/ansible/ansible.git (fetch)
origin  https://github.com/ansible/ansible.git (push)

shell> git branch
* devel

shell> git log --grep="Replace FreeBSD 13.5 with 15.0"

commit 69afa45880c9c03009b178632dccc7c0ffc5b5fa
Author: Matt Clay <matt@mystile.com>
Date:   Wed Jan 7 13:45:42 2026 -0800

    ansible-test - Replace FreeBSD 13.5 with 15.0 (#86385)
```

(9) Ansible collections community.*

Ansible collection community.general is the largest community collection with more than 1000 contributors. At the moment, it comprises 580 modules

```
shell> ansible-doc -t module -l | grep community.general | wc -l
580
```

Supported by Ansible community.

Take a look at what FreeBSD version(s) are being tested.

Quoting CfgMgmtCamp 2026 discussion (8/12): Instant Ansible-test target updates without announcements

> *The VM images are sometimes also changed for stable branches, in the same manner: no announcement, no overlap between availability of the old and new image. As an example, when I woke up on January 8th, I noticed the following replacements since the previous evening: . . . freebsd/13.5 VM → freebsd/15.0*

```
shell> git remote -v
origin  https://github.com/ansible-collections/community.general.gi
origin  https://github.com/ansible-collections/community.general.gi

shell> git branch
* main
```

(10) Ansible collection dedicated to FreeBSD is needed

FreeBSD collection is missing. In a certain sense, this is a good news. Ansible is mature, so we can avoid trial and error when building the Ansible FreeBSD collection.
The question is not `if` FreeBSD collection? But `how`? - this is a proposal - the solution should be as flexible as possible
Proposed collection. There are open questions: - Cover only FreeBSD or all BSD flavours? - What shall be the namespace (freebsd or bsd?)
If there is time we describe the setup of the proposed collection later (Appendix): - setup; start with a minimal tested content; customize your collection on-demand - setup description (distfiles, files, vars, playbooks setup.yml and .configure.yml) The framework is well-known from the ports collection.
When to write a new plugin? - A module vs. command/shell ansible module. For example, iocage is a complex utility. The module is idempotent but slow. Optionally, use the parameter 'creates' to make the command idempotent. This will be faster.
General dilemmas: - Use a module or a command (ansible.builtin.command or ansible.builtin.shell) e.g. iocage? - To configure a file, instead of a modules to run a command you can use modules 'lineinfile' or 'blockinfile'. e.g. sysrc
This is a trade-off: A specific module provides more comfort, but it's slower.

Section 2

FreeBSD collection

(12) Proposed FreeBSD collection

(click at the links)
In this tutorial, we'll be mainly skipping among these 3 sites. - Most information is in the documentation at `readthedocs.io`. - Review the code at `GitHub` if you want to learn more details. - The documentation at `Ansible Galaxy` provides quick reference to the plugins, roles, and playbooks.
Use the command line to get a plugin description fast. For example,

```
shell> ansible-doc -t inventory vbotka.freebsd.iocage
```

(13) Collection Content

(click at the links)

(14) We focus on the iocage plugins

(click at the links)

(15) Plugin inventory iocage

inventory vbotka.freebsd.iocage

See the documentation. There are plenty of options.

```
shell> ansible-doc -t inventory vbotka.freebsd.iocage
```

See the inventory plugins `ansible.builtin.constructed` what options belong to this plugin

```
shell> ansible-doc -t inventory ansible.builtin.constructed
```

See Cache plugins what options belong to cache.

See the source code what options belong to this plugin:

- host
- user
- sudo
- sudo_preserve_env
- get_properties
- env
- hooks_results
- inventory_hostname_tag
- inventory_hostname_required

Side-note: You can use the inventory plugin `community.general.iocage` from the Ansible distribution.

(16) Plugin inventory iocage - Basics

Basics

(17) Plugin inventory iocage - DHCP

DHCP

(18) Plugin inventory iocage - Hooks

Hooks

(19) Plugin inventory iocage - Properties

Properties

(20) Plugin inventory iocage - Tags

Tags

(21) Plugin inventory iocage - Aliases

Aliases

(22) Module iocage

See vbotka.freebsd.iocage - See the description of the parameter `state`. The choices provide the corresponding *iocage* commands. - Without any parameters the module creates *iocage* datasets in the dictionary `ansible_facts` - See, for example, the *sandbox* example 903

(23) Filter iocage

To get the *iocage* datasets, it is faster to: - execute the `iocage list ...` from the module `ansible.builtin.command`, - register the output, and - use the filter `iocage` to parse the output
See, for example, the *sandbox* example 901

Section 3

Examples

(25) Example groups

examples

(26) How to use the examples

(27) Notes

Section 4

Install, configure, and activate iocage

(The below link shows the examples' categories) Manage iocage
Let's start with the first 3 examples to `Manage iocage host`.

(29) example 001: Install Iocage

In the slide, click at the `source code` and `results` links.
The `source code` link shows the directory of this example at GitHub and the `results` link shows the example in the documentation at `ReadTheDocs.io`.
Let's review the role `vbotka.freebsd.iocage`. Open the link `role vbotka.freebsd.iocage`. This is the Ansible Galaxy page of the collection showing the README file of the role `freebsd_iocage`.
Here it is the right time to explain the roles' naming conventions.
All roles included in this collection have got repositories in `GitHub` and are published as standalone roles in `Ansible Galaxy`. For example, this role is named `freebsd_iocage` in the Ansible Galaxy name space `vbotka`. In this collection the role is simply named `iocage`. See the list of the roles at the left side. Clink the link `Ansible role`. This is the standalone role `vbotka.freebsd_iocage` in `Ansible Galaxy`.
`Ansible Galaxy` hosts both standalone roles and collections.
See the Naming conventions in the `Administration guide`. If there is time, we'll explain the setup of the collection later.
You can use the standalone roles or the roles included in the collection. The code is the same.
Take a look at the directory roles
You see what versions of the roles are included in the collection. The names of

(30) example 002: Activate iocage

In the slide, click at the `source code` and `results` links.

Quote Activate iocage:

> *Before iocage is functional, it needs to activate. Essentially, iocage needs to link with a usable zpool.*

The files' tree shows a new directory `host_vars` that keeps the host-specific variables. In production, it is recommended to use a dedicated ZFS pool for iocage, for example, `iocage` in the host `iocage_04`. For testing, you can use `zroot`.

(31) example 003: Audit iocage host

In the slide, click at the `source code` and `results` links.
Use the role `vbotka.freebsd.iocage` to audit the *iocage* configuration.
The installation and configuration of the *iocage* host is described in the details in the example `501`. Here we run the role to make sure everything is configured properly.

Section 5

Plugins iocage

(33) example 010: Clone basejails and create inventory

In the slide, click at the `source code` and `results` links.
The module `vbotka.freebsd.iocage` is used to: - fetch release (Optionally, fetch the releases manually before you run the play.) - create basejail - clone and start jails See the playbook pb-iocage-fetch-base-clone-list.yml
See host_vars In *iocage_02* test `shared IP` networking option. In *iocage_04* test `VNET`. See Networking
See results.

(34) example 011: Display variables iocage_*

In the slide, click at the `source code` and `results` links.
Compare the `Playbook output` with the table `Jails at iocage_04` from the previous example `010`. The variables `iocage_*` are created by parsing the columns of the table. The column `name` is used as the name of the jails. The variable `iocage_name` is **not** created.
The column `IP4` may provide more addresses separated by comma. Therefor the attribute `ip4` of the dictionary `iocage_ip4_dict` is a list. The variable `iocage_ip4` is the address of the first item from this list.
The *iocage* convention is to use dash '-' if a value is missing. For example, `iocage_ip6: -`
See the inventory configuration file iocage.yml The variables `iocage_*` are used to: - `compose` variables: * ansible_host * release * release_major * release_minor - create `keyed_groups`: * distro * state

(35) example 012: Display iocage_properties

In the slide, click at the `source code` and `results` links.
By default (`get_properties: False`) the properties are not provided. See the documentation Properties

(36) example 013: Tags and custom groups

In the slide, click at the `source code` and `results` links.

Tags are key-value pairs for organizing resources

See the documentation Tags

Quoting man iocage:

```
PROPERTIES
...
notes="any string"
    Custom notes for miscellaneous tagging.
    Default: none
    Source: local
```

(37) example 014: Inventory cache

In the slide, click at the `source code` and `results` links.
Compare the timing Clear cache Cache enabled

(38) example 015: Multiple inventory cache

In the slide, click at the `source code` and `results` links.
The parameter cache_prefix must be used if there are more inventory sources.

(39) example 016: Multiple inventory constructed

In the slide, click at the `source code` and `results` links.
The inventory plugin ansible.builtin.constructed must be used to create group from all jails

(40) example 017: community.general.iocage

In the slide, click at the `source code` and `results` links.
The inventory plugin community.general.iocage is included in the Ansible collection `community.general`
This plugin can be used in a standard Ansible installation without any requirements. See:

```
shell> ansible-doc -t inventory community.general.iocage
```

(41) example 018: Clone basejails. Use DHCP.

In the slide, click at the `source code` and `results` links.
The module `vbotka.freebsd.iocage` is used to: - clone and start jails See the playbook pb-iocage-clone-list.yml
See the `properties` in host_vars
Quoting man iocage:

```
PROPERTIES
...
dhcp=[1 | 0] This controls starting the jail with the Dynamic Host
             Configuration Protocol enabled. To enable dhcp, vnet
             bpf must also be enabled.
             Default: 0
             Source: local
```

See the results.

(42) example 019: Inventory option use_vars_plugins

In the slide, click at the `source code` and `results` links.
Despite the fact that inventory plugin `iocage` extends `constructed` See source code

```
extends_documentation_fragment:
  - ansible.builtin.constructed
  - ansible.builtin.inventory_cache
```

The option use_vars_plugins is not available in the inventory plugin `iocage`
See the results.

(43) example 020: Get inventory aliases from notes

In the slide, click at the `source code` and `results` links.
Use the option inventory_hostname_tag to tell the plugin which tag to use.
See Aliases

(44) example 030: Create custom facts

In the slide, click at the `source code` and `results` links.

Create custom facts to provide a dictionary of iocage datasets lists.

See Adding custom facts

As a side note 1: The module ansible.builtin.meta provides option `refresh_inventory` quoting:

```
forces the reload of the inventory, which in the case of dynamic inv
If the dynamic inventory script is using a cache, Ansible cannot kno
This is mainly useful when additional hosts are created and users wi
ansible.builtin.add_host module.
```

As a side note 2: The task

```
    - name: Get custom facts
      ansible.builtin.setup:
        filter: ansible_local
```

can be used instead of this task

```
    - name: Get iocage datasets
      vbotka.freebsd.iocage:
```

Section 6

Ansible client

(46) example 200: Create iocage templates. Clone jails.

In the slide, click at the `source code` and `results` links.
See the playbook: pb_iocage_template
See the source code: pb_iocage_template tasks
The most time-consuming part is the installation of the packages:

- See the task create.yml Either use the attribute *pkglist* in the module *vbotka.freebsd.iocage*
- See the task pkg.yml Or, use the attributes *jail* and *name* in the module *community.general.pkgng*
- See the NOTES

Note: The module *community.general.pkgng* is *jail-aware*
See: community.general.pkgng jail aware
See the playbook: pb_iocage_ansible_clients
See the source code: pb_iocage_ansible_clients.yml tasks
Notes:

- If possible, install packages in a standalone tasks. Then, disable the installation to speedup the plays.

(47) example 201: Display iocage datasets

In the slide, click at the `source code` and `results` links.
See the playbook: pb-iocage-display-datasets.yml
There are 4 datasets created by *iocage*:

- releases
- plugins
- templates
- jails

In the first task, the module *vbotka.freebsd.iocage* without arguments creates the datasets. The datasets are stored in the dictionary *ansible_facts*.
For abbreviation, the *properties* are removed from *templates* and *jails*.

(48) example 202: Create iocage templates. Clone DHCP jails.

In the slide, click at the `source code` and `results` links.
In addition to the `example 200` we configure `dhclient hooks`. Then, when we clone the jails, DHCP is used to assign the addresses. See the *host_vars*. The common *properties* are used.
Explanation of the warning: "The user act_user must exist on the iocage host. Otherwise, the module ansible.posix.authorized_key will crash."
In the module *ansible.posix.authorized_key* we configure the path for each jail. The module assumes the keys belongs to a user on the iocage host. The module is checking whether the user exists even when the parameter *path* is used.

```
- ansible.posix.authorized_key:
    user: "{{ _act_user }}"
    path: "{{ freebsd_iocage_mount }}/jails/{{ item.key }}/root/home
    ...
  loop: "{{ templates | dict2items }}"
```

(49) example 203: Create DHCP jails with auto UUID and iocage_tags

In the slide, click at the `source code` and `results` links.
We use here the templates created in the previous `example 202`.
iocage is able to create more jails with the same parameters if the option *–count* is used. In this case, it's more efficient to use the module *ansible.builtin.command* instead of the module *iocage*, because such a step is not idempotent anyway. See the examples in the `Use case`.
Here we use the playbook `pb_iocage_ansible_clients.yml` again, but, this time, to create *swarms*.
See swarms.
We borrowed *swarms* from the "Container swarms", which refer to a cluster of machines configured to work together as a single, unified system. It is not related to Docker in any way.
See the source code: pb_iocage_ansible_clients.yml tasks
Let's take a look at the tasks in swarms.yml
The first task `vbotka.freebsd.iocage` creates the *iocage* datasets only.
In the dictionary `swarms` the parameter `count` declares the number of jails in each swarm. We want to keep this number of jails. In the playbook section `vars`, the dictionary `_tags` is declared.
As a side-note: This is a very useful construct, frequently used to create

(50) example 204: Create DHCP jails with auto UUID and iocage_tags v2

In the slide, click at the `source code` and `results` links.
Also here we use the templates created in the `example 202`. Compared with `example 203`, the difference is using the filter `iocage` instead of the module `iocage` to create the dictionary `iocage_jails`.
Take a look at the *tree*. Instead of the collection playbook `vbotka.freebsd.pb_iocage_ansible_clients.yml` we use the modified playbook `pb-iocage-ansible-clients-v2.yml` and the tasks
In the tasks swarm.yml we import iocage_jails.yml instead of the module `iocage` which creates the dictionary `iocage_jails`.
If interested, take a look at `iocage_jails.yml` how the dictionary is created.
If the jails are present, for example,

```
shell>  iocage list
+-----+----------+-------+--------------+------+
| JID |   NAME   | STATE |   RELEASE    | IP4  |
+=====+==========+=======+==============+======+
| 1   | cb040et9 | up    | 15.0-RELEASE | DHCP |
+-----+----------+-------+--------------+------+
| 2   | dd911c4f | up    | 15.0-RELEASE | DHCP |
+-----+----------+-------+--------------+------+
```

(51) example 206: Create DHCP and fixed IP jails

In the slide, click at the `source code` and `results` links.
This use case demonstrates the **advantage of silently ignoring failed items** over the potential explicit error handling. Let the option compose pick what is needed.

(52) example 207: Create DHCP jails with auto UUID, iocage_tags, alias and class

In the slide, click at the `source code` and `results` links.
Use the option `inventory_hostname_tag` to create inventory aliases.
Quoting Inventory aliases

> *The inventory_hostname is the unique identifier for a host in Ansible. This identifier can be an IP address or a hostname, but it can also be just an 'alias' or short name for the host.*

We use *swarms* here to create jails. Hence, the jails' names will be random. In the inventory we will:

- create `aliases`, declared in the dictionary `project`, and
- compose variables `ansible_host`

The goal is to efficiently create jails by the *iocage* `--count` option and still have dedicated jails' names. We also declare the `class` tag that will be used later (example 500).

(53) example 208: Create iocage template for ansible-pull

In the slide, click at the `source code` and `results` links.
In addition to the template `ansible_client`, 2 more packages are installed in the template `ansible_client_pull`:

- git
- py311-ansible

These packages are needed to

> *pull a remote copy of ansible on each managed node ... and update playbook source via a source repository.*

An example how to use the template `ansible_client_pull` is not ready yet.

(54) example 209: Create iocage pkglist file

In the slide, click at the `source code` and `results` links.
Explain `Notes`, `group_vars`, and `host_vars`.

(55) example 210: Test empty iocage notes

In the slide, click at the `source code` and `results` links.
This is a trivial case of test empty iocage notes. Create iocage_tags. The result should be an empty dictionary.

SECTION 7

Infrastructure

(57) example 500: syslog-ng server and syslog-ng clients

In the slide, click at the `source code` and `results` links.
The project keys are jail's aliases.

(58) example 501: iocage host

In the slide, click at the `source code` and `results` links.
Configure iocage host.

(59) example 502: Branch server

In the slide, click at the `source code` and `results` links.
WIP

(60) example 510: Project ansible-pull

In the slide, click at the `source code` and `results` links.
WIP

Section 8

Appendix

(62) App1. FreeBSD collection configuration

(63) App2. FreeBSD collection setup

(64) App3. Recommended reading

EOF